

您的文档标题

作者姓名

2026 年 06 月 17 日

1 chapter10: 随机算法

1.1 10.1 引言

1.1.1 10.1.1 随机化算法

随机化算法把“对于所有合理的输入都必须给出正确的输出”这一求解问题的条件放宽，将随机性的选择注入到算法中，在算法执行某些步骤时可以随机地选择下一步该如何进展，同时允许结果以较小的概率出现错误，并以此为代价获得算法运行时间的大幅度减少。

随机化算法优点

- 与确定性算法相比，随机算法所需运行时间或空间通常较小；
- 随机算法易于理解和实现。

1.1.2 10.1.2 两类随机算法及其关系

Las Vegas 算法（拉斯维加斯算法）¹

- 总是给出正确的解，或者无解。
- 运行时间本身是一个随机变量，通常需要分析算法的平均运行时间。
- 期望运行时间是输入规模的多项式且总能给出正确答案的随机算法称为**有效的拉斯维加斯算法**，也称为 **ZPP 类算法** (zero-error probabilistic polynomial time)，即零错误概率多项式时间随机算法。

Monte Carlo 算法（蒙特卡罗算法）

- 总是给出解，但偶尔可能是错误解。
- 可以通过多次运行原算法，且满足每次运行时随机选择都相互独立，使产生错误解的概率减到任意小。
- 运行时间和出错概率都是随机变量。
- 总是在多项式时间内运行且出错概率不超过 $\frac{1}{3}$ （可压到任意小）的随机算法称为**有效的蒙特卡罗算法**，也称为 **BPP 类算法** (bounded-error probabilistic polynomial time)，即错误概率有限的多项式时间随机算法。

错误类型：

- **弃真型错误**：在求解判定问题时把本应接受的输入误判为拒绝。
- **取伪型错误**：把本应拒绝的输入误判为接受。

根据是否同时出现两类错误，Monte Carlo 算法又分为：

1. **单侧错误的 Monte Carlo 算法**：在所有输入上只犯弃真或只犯取伪的错误。

¹Las Vegas 和 Monte Carlo 不是一种算法的名称，而是对一类随机算法的特性的概括。不太可能为 NP 完全问题设计出多项式时间的随机算法

- 有效的**弃真型**单侧错误随机算法称为 **RP 类算法** (Randomized Polynomial-time)
 - 有效的**取伪型**单侧错误随机算法称为 **coRP 类算法** (Complementary Randomized Polynomial-time, RP 的补类)
2. **双侧错误**的 Monte Carlo 算法: 在所有输入上可以同时出现两种不同的错误。

Las Vegas 算法与 Monte Carlo 算法的对比:

比较项目	Las Vegas	Monte Carlo
规律	采样越多越有可能找到最优解	采样越多越逼近最优解
例子	从一串钥匙中试出能开锁的钥匙	从不透明的苹果筐中挑最大的苹果
策略	尽量找最好的, 但不保证能找到 (除非全枚举)	尽量找好的, 但不保证是最好
适用情景	要求必须给出最优解	要求在有限采样内必须给出一个解, 但不要求最优

1.1.3 10.1.3 随机算法的时间复杂度分析

对于随机算法, 在一个大小为 n 的固定实例 I 上的运行时间, 一次和另一次的执行可能不相同。因此, 一种更自然的度量是算法 A 在固定实例 I 上的**期望运行时间**——用算法 A 反复求解实例 I 的平均时间。

- 对于**确定性算法**, 求解期望运行时间时, 概率分布定义在**输入**上。
- 对于**随机算法**, 求解期望运行时间时, 概率分布定义在**算法** (随机数序列) 上。

预期时间 (Expected Time): 对于大小为 n 的特定输入 I , 算法的预期时间就是在不同随机数序列上得到的平均时间:

$$ET(A, I) = \sum_{\text{random sequence } R} \text{probability}(R) * \text{time}(A, I, R)$$

最坏情况预期时间 (Worst-case Expected Time): 算法的最坏情况预期时间是所有长度为 n 的输入的预期时间的最大值:

$$W_ET(A) = \max_{\text{input } I \text{ with size } n} \left[\sum_{\text{random sequence } R} \text{probability}(R) * \text{time}(A, I, R) \right]$$

核心区别: 确定性算法对输入取平均 (假设均匀分布), 随机算法对随机数取平均 (输入固定)。因此随机算法的复杂度分析不依赖输入分布假设。

1.2 10.2 随机算法举例

这是一个很不错的讲解栏目：<https://www.luogu.com.cn/article/wegx023s>

1.2.1 10.2.1 素数测试

1.2.1.1 问题背景

直观的试除法（用2到 $\sqrt{n}$ 的数去除）效率极低——因数分解远难于素性测试本身。随机算法的突破口：找**合数的证据**（witness）而非直接分解因子。

1.2.1.2 模幂运算：ExpMod

设 a, m, n 为正整数且 $m \leq n$ ，计算 $a^m \pmod n$ 。算法在每次平方或乘法之后取模，而非先算 a^m 再取模。

设 m 的二进制数字为 $b_k, b_{k-1}, \dots, b_0$

$c \leftarrow 1$

for $j \leftarrow k$ downto 0:

$c \leftarrow c^2 \pmod n$

 if $b_j = 1$ then $c \leftarrow ac \pmod n$

return c

如果每次乘法花费1个时间单元，运行时间为 $\Theta(\log n)$ 。但如果处理任意大整数，两数相乘为 $O(\log^2 n)$ ，则总运行时间为 $O(\log^3 n)$ 。

1.2.1.3 费马小定理

定理：如果 n 是素数，那么对于所有 $a \not\equiv 0 \pmod n$ ，有

$$a^{n-1} \equiv 1 \pmod n$$

逆否命题：若存在 a 使得 $a^{n-1} \not\equiv 1 \pmod n$ ，则 n 一定是合数（ a 称为 n 的见证者）。但**费马定理的逆不成立**——存在合数 n 满足 $a^{n-1} \equiv 1 \pmod n$ ，称为基 a 的**伪素数**。

1.2.1.4 PTEST1：固定底数测试

固定测试底数 $a = 2$ ：

输入：正奇整数 $n \geq 5$

输出：若 n 是素数则返回 prime（可能），否则返回 composite（确定）

if $\text{ExpMod}(2, n-1, n) \equiv 1 \pmod n$ then return prime 可能
else return composite 确定

缺点：存在基 2 的伪素数。例如 $341 = 11 \times 31$ 满足 $2^{340} \equiv 1 \pmod{341}$ 。在 4 到 2000 之间的合数中，仅对 341, 561, 645, 1105, 1387, 1729, 1905 这 7 个误判。小于 100,000 的数中仅 78 个出错，最大的是 $93,961 = 7 \times 31 \times 433$ 。

1.2.1.5 PTEST2：随机底数的费马测试

改进：在 $[2, n-2]$ 之间随机选择底数 a ：

输入：正奇整数 $n \geq 5$

输出：若 n 是素数则返回 prime（可能），否则返回 composite（确定）

$a \leftarrow \text{random}(2, n-2)$

if $\text{ExpMod}(a, n-1, n) \equiv 1 \pmod{n}$ then return prime 可能

else return composite 确定

引理：如果 n 不是 Carmichael 数，则 PTEST2 检测出 n 为合数的概率至少为 $\frac{1}{2}$ 。

证明思路：设 Z_n^* 为小于 n 且与 n 互素的正整数集合，定义

$$F_n = \{a \in Z_n^* \mid a^{n-1} \equiv 1 \pmod{n}\}$$

若 n 既不是素数也不是 Carmichael 数，则 F_n 是 Z_n^* 的真子群。由拉格朗日定理， $|F_n|$ 整除 $|Z_n^*| = \varphi(n)$ ，从而 $|F_n| \leq \frac{1}{2} |Z_n^*|$ 。即 Z_n^* 中至少有一半的元素可作为合数的见证者。

1.2.1.6 卡迈克尔数（Carmichael 数）

卡迈克尔数是一类特殊的合数——它们对所有与 n 互质的 a 均满足费马定理 $a^{n-1} \equiv 1 \pmod{n}$ 。因此用费马测试无法检出。

最小的 Carmichael 数： $561 = 3 \times 11 \times 17$ ， $1105 = 5 \times 13 \times 17$ ， $1729 = 7 \times 13 \times 19$ ， $2465 = 5 \times 17 \times 29$ 。

在小于 10^8 的范围内仅有 255 个。但已证明存在无穷多个 Carmichael 数。这促使了更强测试的诞生。

1.2.1.7 PTEST3：非平凡平方根检测

PTEST3 的核心思想是：在素数模 n 下，方程 $x^2 \equiv 1 \pmod{n}$ 的**唯一解**只有 $x \equiv 1$ 和 $x \equiv -1 \pmod{n}$ （即 $n-1$ ）。这两个解称为**平凡根**。任何其他根都是**非平凡平方根**，其存在是 n 为合数的**铁证**。

算法的构造：将 $n-1$ 分解为 $m \cdot 2^q$ （ m 为奇数， $q \geq 1$ ）。然后构造序列：

$$a^m \pmod{n}, a^{2m} \pmod{n}, a^{4m} \pmod{n}, \dots, a^{2^q m} \pmod{n}$$

注意 $a^{2^q m} = a^{n-1}$ ，因此第 q 步正好就是 $a^{n-1} \pmod{n}$ ——费马定理的检查点。沿途每一步检查：若 $x_i = 1$ 但 $x_{i-1} \neq 1$ 且 $x_{i-1} \neq n-1$ ，则找到了非平凡平方根， n 必为合数。

算法 PTEST3(n, q, m) $a \leftarrow \text{random}(2, n - 1)$
 $x_0 \leftarrow \text{ExpMod}(a, m, n)$
for $i \leftarrow 1$ to q :
 $x_i \leftarrow x_{i-1}^2 \pmod n$
 如果 $x_i = 1$ 且 $x_{i-1} \neq 1$ 且 $x_{i-1} \neq n - 1$
 则返回 composite 肯定
if $x_q \neq 1$ then return composite 肯定
return prime 可能

条件拆解：三种可能情况

场景	x_{i-1}	$x_i = x_{i-1}^2$	结论
正常①	1	$1^2 = 1$	✔ 前一步已是1, 正常
正常②	$n - 1 (= -1)$	$(-1)^2 = 1$	✔ 素数允许 $-1 \rightarrow 1$
合数铁证	既不是1也不是 $n - 1$	某数的平方 $\equiv 1$	✘ 找到了非平凡平方根, n 必为合数

关于本题中参数 q 的作用：因 $n - 1 = m \cdot 2^q$ ，故第 q 步 $x_q = a^{2^q m} = a^{n-1} \pmod n$ ，后续再平方无意义（ $1^2 = 1$ 死循环）。PTEST3 在路径的每个平方步骤设置哨兵，比只检查终点 a^{n-1} 的 PTEST2 多了一层非平凡平方根检测，因此能绕过 Carmichael 数。

1.2.1.8 Miller-Rabin 算法 (PrimalityTest)

PTEST3 单次测试可能出错（把合数判为素数）的概率至多为 $\frac{1}{2}$ 。通过重复独立测试 k 次，错误概率降至 2^{-k} 。

PrimalityTest：PTEST3 的 k 次重复版本

输入：正奇整数 $n \geq 5$
输出：若 n 是素数则返回 prime，否则返回 composite
分解 $n - 1 = m \cdot 2^q$ ， m 为奇数
 $k \leftarrow \lceil \log n \rceil$
for $i \leftarrow 1$ to k :
 $a \leftarrow \text{random}(2, n - 2)$
 调用 PTEST3(n, q, m)，若返回 composite 则直接输出 composite 确定
return prime 可能

错误概率：取 $k = \lceil \log n \rceil$ ，失败概率

$$2^{-\lceil \log n \rceil} \leq \frac{1}{n}$$

即算法以至少 $1 - \frac{1}{n}$ 的概率给出正确回答。当 n 足够大时，错误概率可忽略。

算法层次关系：

PTEST1 $\subset$ PTEST2 $\subset$ PTEST3 $\subset$ PrimalityTest

PTEST1: 固定底 $a = 2$ 的费马测试（受限于基 2 伪素数）

PTEST2: 随机底的费马测试（受限于 Carmichael 数）

PTEST3: 核心子程序，反复平方检测非平凡平方根（单次错率 $\leq \frac{1}{2}$ ）

PrimalityTest: PTEST3 重复 $\lceil \log n \rceil$ 次（总错率 $\leq \frac{1}{n}$ ）

时间复杂度：总时间 $O(\log^4 n)$ 。

1.2.2 10.2.2 测试串的相等性

1.2.2.1 问题与指纹思想

A 组有串 x , B 组有串 y , 通过一条窄信道通信, 要判断 $x = y$ 是否成立。

朴素方案: A 将整个 x 传过去——浪费带宽。指纹方案: A 从一个短得多的串作为 x 的“指纹”, 传给 B; B 用同样方法获得 y 的指纹。指纹相等则假定 $x = y$, 否则得出 $x \neq y$ 。

指纹函数：选一个素数 p , 定义

$$I_{p(x)} = I(x) \bmod p$$

其中 $I(x)$ 是比特串 x 表示的整数。若 p 不太大, $I_{p(x)}$ 只需 $O(\log p)$ 比特传输。

1.2.2.2 假匹配与失败概率

若 $x \neq y$ 但 $I_{p(x)} = I_{p(y)}$, 即 p 整除 $I(x) - I(y)$, 称为**假匹配**。

由**素数定理**: $\pi(x)$ (不超过 x 的素数个数) 满足

$$\pi(x) \sim \frac{x}{\ln x}$$

即素数密度约 $\frac{1}{\ln x}$ 。这意味着从 $[1, M]$ 中随机选素数, 撞上特因子的概率可控。

设串 x 和 y 的长度均为 n 比特。对随机选定的素数 $p \leq M$, 假匹配概率为:

$$\Pr[\text{失败}] = \frac{|\{p \leq M : p \mid (I(x) - I(y))\}|}{\pi(M)} \leq \frac{\pi(n)}{\pi(M)}$$

取 $M = 2n^2$, 则

$$\Pr[\text{失败}] \leq \frac{\pi(n)}{\pi(2n^2)} \approx \frac{\frac{n}{\ln n}}{2 \frac{n^2}{\ln(2n^2)}} = \frac{\ln(2n^2)}{2n \ln n} \approx \frac{1}{n}$$

1.2.2.3 重复执行降低错误

重复算法 k 次（每次独立选择新素数 p ）：

$$\Pr[\text{失败}] \leq \left(\frac{1}{n}\right)^k$$

取 $k = \lceil \log \log n \rceil$ 时，失败概率降至 $n^{-\lceil \log \log n \rceil}$ 。

1.2.2.4 实例计算

当 $n = 1,000,000$ （100 万比特串）：

- $M = 2 \times 10^{12} = 2^{40.8631}$
- 传输 p 最多需要 $\lfloor \log M \rfloor + 1 = 40 + 1 = 41$ 比特
- 传输指纹最多 ≤ 41 比特
- 总传输量：**82 比特**（对比传输原串 100 万比特）
- 一次传输失败概率： $\frac{1}{n} = 10^{-6}$
- 重复 5 次后： $(10^{-6})^{-5} = 10^{-30}$ （几乎不可能发生）

1.2.3 模式匹配

1.2.3.1 滚动哈希

将文本 $X = x_1x_2 \cdots x_n$ 与模式 $Y = y_1y_2 \cdots y_m$ （ $m \leq n$ ）做匹配。利用随机算法比较模式指纹 $I_{p(Y)}$ 和文本块指纹 $I_{p(X(j))}$ （其中 $X(j) = x_j \cdots x_{j+m-1}$ ）。

将子串视为二进制表示的整数，则从窗口 j 滑动到窗口 $j+1$ 只需三步：

1. 去掉最高位 $x_j \cdot 2^{m-1}$
2. 整体左移一位（乘以 2）
3. 加上新低位 x_{j+m}

约去模 p 后得滚动哈希公式：

$$I_{p(X(j+1))} \equiv (2 \cdot I_{p(X(j))} - 2^m x_j + x_{j+m}) \bmod p$$

记 $W_p = 2^m \bmod p$ （只需预计算一次）：

$$I_{p(X(j+1))} \equiv (2I_{p(X(j))} - W_p x_j + x_{j+m}) \bmod p$$

1.2.3.2 算法流程

从小于 M 的素数集中随机选择 p

$j \leftarrow 1$

计算 $W_p = 2^m \pmod p$; $I_p(Y)$; $I_p(X(1))$
 while $j \leq n - m + 1$:
 if $I_p(X(j)) = I_p(Y)$ then return j (可能找到匹配)
 用滚动哈希公式计算 $I_p(X(j+1))$
 $j \leftarrow j + 1$
 return 0 (确定 Y 不在 X 中)

1.2.3.3 复杂度与错误分析

- 计算 $W_p, I_p(Y), I_p(X(1))$: $O(m)$
- 每个后续窗口: $O(1)$ (常数次加减模运算)
- 总时间: $O(n + m)$

假匹配概率: 若 $Y \neq X(j)$ 但 $I_p(Y) = I_p(X(j))$, 则 p 整除

$$\prod_{\{j \mid Y \neq X(j)\}} |I(Y) - I(X(j))|$$

该乘积不超过 2^{mn} , 整除它的素数个数不超过 $\pi(mn)$ 。取 $M = 2mn^2$ 时:

$$\Pr[\text{假匹配}] \leq \frac{\pi(mn)}{\pi(2mn^2)} \approx \frac{1}{n}$$

关键结论: 假匹配概率仅与文本长度 n 有关, 与模式长度 m 无关。

1.2.3.4 转换为 Las Vegas 算法

当指纹匹配时, 额外进行一次逐字符确认 ($O(m)$ 时间)。期望时间仍为 $O(n + m)$ 。

1.2.4 10.2.4 货郎担问题 (TSP)

1.2.4.1 问题描述

旅行商人要拜访 n 个城市, 每个城市只能拜访一次, 最后回到出发城市。目标: 选择总路程最短的哈密顿回路。

1.2.4.2 随机采样算法 (Monte Carlo)

```
bestPath  $\leftarrow$  []
bestDistance  $\leftarrow$   $\infty$ 
path  $\leftarrow$  随机排列(1, 2, ..., n)
distance  $\leftarrow$  计算路径总长度
for  $i \leftarrow 1$  to 100000:
  if distance < bestDistance:
    bestPath  $\leftarrow$  path
    bestDistance  $\leftarrow$  distance
```

重新生成随机路径并计算距离

return bestPath, bestDistance

这是一个 **Monte Carlo 算法**：一定有输出，但未必是最优解。随着采样次数增加，找到最优的概率增大。

局限性： n 个城市的哈密顿回路总数为 $\frac{(n-1)!}{2}$ 。采样 10 万次时， $n = 10$ 时尚可覆盖约 55%，但 $n = 15$ 时空间可达约 870 亿，采样几乎无法覆盖。因此纯随机采样在大规模 TSP 上不实用。实际中常用模拟退火、遗传算法、随机局部搜索等更智能的随机化策略。

1.2.5 10.2.5 随机游走算法

1.2.5.1 基本概念

随机游走 (Random Walk) 由 Karl Pearson 于 1905 年提出，源于物理学中的布朗运动模型。在计算机科学中，随机游走用于在图上进行关系传递分析。

基本规则：从一个或一系列顶点开始遍历一张图。在任意顶点，遍历者以概率 $1 - a$ 游走到一个邻居，以概率 a 随机跳转到图中的任意顶点 (a 称为跳转概率)。反复迭代后概率分布收敛于平稳分布。著名应用：PageRank 算法。

1.2.5.2 一维有边界随机游走的数学推导

问题设定：一维数轴上，初始位置 $x = n$ ，左右吸收壁为 $x = 0$ 和 $x = w$ ($0 \leq n \leq w$, n, w 为整数)。每单位时间向左/向右移动 1 步的概率均为 $\frac{1}{2}$ ，到达吸收壁后停止。

定义 S_n ：从位置 n 出发最终落入左边界 $x = 0$ 的概率。边界条件： $S_0 = 1$ (已在左边)， $S_w = 0$ (在右边不可能落入左边)。

对中间状态 $0 < n < w$ ，由全概率公式：

$$S_n = \frac{1}{2}S_{n-1} + \frac{1}{2}S_{n+1}$$

两边乘以 2 并移项：

$$2S_n = S_{n-1} + S_{n+1}$$

$$S_{n+1} - S_n = S_n - S_{n-1}$$

相邻差为常数， S_n 是等差数列。设 $S_{n+1} - S_n = k$ ，则

$$S_n = kn + S_0$$

代入 $S_0 = 1, S_w = 0$ 得 $k = -\frac{1}{w}$ ，因此：

$$S_n = 1 - \frac{n}{w} = \frac{w-n}{w}$$

数值感受：本金 $n = 100$ ，目标 $w = 200$ 时，输光概率 $S_{100} = \frac{200-100}{200} = \frac{1}{2}$ 。若目标 $w = 300$ ，则输光概率 $S_{100} = \frac{300-100}{300} = \frac{2}{3}$ ——目标越远，输光概率越大。

1.2.5.3 单边界情况：酒鬼必掉悬崖

将右边界推至无穷远 $w \rightarrow +\infty$ ：

$$S_n = \lim_{w \rightarrow +\infty} \frac{w - n}{w} = 1$$

从任何有限位置出发，最终落入左边界的概率为1。**酒鬼失足问题：**酒鬼离悬崖仅一步之遥，每步前后各 $\frac{1}{2}$ ，最终必定掉下悬崖。**赌徒输光问题：**在公平赌博中，赌徒面对庄家无穷资本时，迟早会输光所有赌金。

1.2.5.4 求解函数全局最优

给定初始迭代点 x ，初次行走步长 λ ，控制精度 ε

while $\lambda > \varepsilon$:

$k \leftarrow 1$

while $k < N$:

随机生成 $(-1, 1)$ 间的 n 维向量 $u = (u_1, \dots, u_n)$

标准化： $u' = \frac{u}{\sqrt{\sum u_i^2}}$ （单位方向向量）

$x_1 \leftarrow x + \lambda u'$

if $f(x_1) < f(x)$:

$k \leftarrow 1$

$x \leftarrow x_1$

else: $k \leftarrow k + 1$

$\lambda \leftarrow \frac{\lambda}{2}$

return x

若连续 N 次找不到更优值，则认为当前最优解在以 λ 为半径的 n 维球内；此时若 $\lambda < \varepsilon$ 则结束，否则步长减半继续。

1.3 知识延展（教材补充）

1.3.1 随机化快速排序

在快速排序的基础上增加**随机选取**的步骤：随机选择区间 $[low, high]$ 中的一个索引 v 作为主元。

过程 $rquicksort(low, high)$

if $low < high$:

$v \leftarrow \text{random}(low, high)$

互换 $A[low]$ 和 $A[v]$

$SPLIT(A[low \dots high], w)$

$rquicksort(low, w - 1)$

$rquicksort(w + 1, high)$

最坏情况运行时间仍为 $\Theta(n^2)$ ，但这与输入形式无关，只取决于随机数生成器的“不幸”。没有一种输入的排列可以引起它的最坏情况。期望运行时间为 $\Theta(n \log n)$ ，是一个 ZPP 类算法。

1.3.2 随机化选择算法

随机选择主元进行划分，期望运行时间 $O(n)$ 且常数因子很小。

过程 $\text{rselect}(A, \text{low}, \text{high}, k)$

$v \leftarrow \text{random}(\text{low}, \text{high})$

$x \leftarrow A[v]$

将 $A[\text{low} \dots \text{high}]$ 分成三个数组：

$A_1 = \{a \mid a < x\}$

$A_2 = \{a \mid a = x\}$

$A_3 = \{a \mid a > x\}$

case:

$|A_1| \geq k$: return $\text{rselect}(A_1, 1, |A_1|, k)$

$|A_1| + |A_2| \geq k$: return x

$|A_1| + |A_2| < k$: return $\text{rselect}(A_3, 1, |A_3|, k - |A_1| - |A_2|)$

期望时间分析：设 $C(n)$ 为 n 个元素上的期望比较次数。通过归纳法证明 $C(n) < 4n$ ，故期望时间为 $\Theta(n)$ 。

1.3.3 多选问题

设有 n 个元素的数组 A ， $K[1 \dots r]$ ($1 \leq r \leq n$) 是由 1 到 n 间 r 个正数构成的有序数组。**多选问题**就是对于所有 i ，选择 A 中的第 $K[i]$ 小元素。若 $r = 1$ 即选择问题， $r = n$ 即排序问题。

方法：随机挑选一个元素 a ，将数组 A 关于 a 分割。若两部分均含靶标则递归求解两部分，否则丢弃不含靶标的部分。

该算法以概率 $1 - O(\frac{1}{n})$ 的时间复杂度为 $O(n \log r)$ 。

1.3.4 随机取样

从 n 个元素的集合中随机选取 m 个元素 ($m < n$)。

初始化布尔数组 $S[1 \dots n]$ 全为 false

$k \leftarrow 0$

while $k < m$:

$r \leftarrow \text{random}(1, n)$

if not $S[r]$:

$k \leftarrow k + 1$

$A[k] \leftarrow r$

$S[r] \leftarrow \text{true}$

若 $m > \frac{n}{2}$ ，反面考虑：随机选 $n - m$ 个整数丢弃，其余保留。

期望时间: 第 $k+1$ 次成功选到新元素的概率 $p_k = \frac{n-k}{n}$ 。由几何分布可知期望尝试次数 $E_k = \frac{n}{n-k}$ 。总期望时间:

$$E[T] = \sum_{k=0}^{m-1} \frac{n}{n-k} = n(H_n - H_{n-m})$$

其中 H_n 为调和数。当 $m = \Theta(n)$ 时 $E[T] = \Theta(n)$ 。

1.3.5 二元布尔可满足问题 (2-SAT)

1.3.5.1 随机游走算法求解 2-SAT

随机初始化赋值 σ : 对每个变元 x_i 随机赋“True”或“False”

for $i \leftarrow 1$ to m :

if σ 满足所有子句 then return σ

任选一个不满足的子句

随机翻转该子句中的一个文字

若 m 次内未找到则重新初始化

定理: 若输入公式是不可满足的, 则算法正确宣布不可满足; 若输入公式是可满足的, 则算法以 $1 - \frac{1}{2^m}$ 概率找到可满足赋值。

1.3.5.2 马尔可夫链分析

定义汉明距离 $d(\sigma, \sigma^*)$ 为当前赋值与最优赋值之间取值不同的变量个数。对每个不满足子句, 至少以 $\frac{1}{2}$ 概率减少汉明距离 (因子句最多含2个文字)。

定义马尔可夫链 $\{Y_0, Y_1, Y_2, \dots\}$:

$$P(Y_{t+1} = j+1 \mid Y_t = j) = P(Y_{t+1} = j-1 \mid Y_t = j) = \frac{1}{2}$$

Y_t 比 X_t 花费更长的期望时间到达状态 n 。设 h_j 为从 j 到达 n 的期望时间, 则:

$$h_n = 0, \quad h_0 = h_1 + 1, \quad h_j = \frac{h_{j+1} + h_{j-1}}{2} + 1$$

由归纳法可得 $h_j \leq n^2 - j^2 \leq n^2$ 。

由马尔可夫不等式 ($P(X \geq \varepsilon) \leq \frac{E[X]}{\varepsilon}$):

$$P(\text{从 } X_0 \text{ 到达 } n \text{ 的时间} \geq 2n^2) \leq \frac{n^2}{2n^2} = \frac{1}{2}$$

经过 $2n^2m$ 步迭代后仍未到达 σ^* 的概率:

$$P(\text{失败}) \leq \left(1 - \frac{1}{2n}\right)^{2n^2m} \approx e^{-nm} \leq \frac{1}{2^m}$$

1.3.5.3 马尔可夫不等式

设 X 为仅取非负的随机变量，数学期望 $E[X] = \mu$ 。则对任意 $\varepsilon > 0$ ：

$$P(X \geq \varepsilon) \leq \frac{\mu}{\varepsilon}$$

证明：定义指示变量 $Y = \varepsilon \cdot \mathbf{1}_{\{X \geq \varepsilon\}}$ ，则 $Y \leq X$ ，故 $E[Y] = \varepsilon P(X \geq \varepsilon) \leq E[X]$ 。

1.3.5.4 马尔可夫链基础

马尔可夫特性 (无后效性)：随机过程在给定 t 时刻的状态后，后续状态及其概率与 t 之前的状态无关。当前状态包含了所有历史信息。

离散随机序列 $\{X_0, X_1, \dots\}$ 若满足

$$P(X_{t+1} = x_{t+1} \mid X_t = x_t, \dots, X_0 = x_0) = P(X_{t+1} = x_{t+1} \mid X_t = x_t)$$

则称为**有限马尔可夫链**。

一步转移矩阵 $P = [p_{i,j}]$ ，其中 $p_{i,j} = P(X_{t+1} = j \mid X_t = i)$ 。每行元素之和为 1。

m 步转移矩阵满足 $P^{(m)} = P^m$ ，状态分布 $\mathbf{p}(t+m) = \mathbf{p}(t)P^m$ 。

1.4 本章作业

1.4.1 优惠券收集问题

有 n 种优惠券，每种数量不限。每次实验随机抽取一张，各概率均等。计算集齐所有 n 种所需的期望次数。

设 X_i 为已收集 $i-1$ 种后首次抽到第 i 种新券所需的次数。已收集 k 种时抽到新券的概率 $p_k = \frac{n-k}{n}$ ， X_{k+1} 服从几何分布，期望 $E[X_{k+1}] = \frac{1}{p_k} = \frac{n}{n-k}$ 。

总次数 $T = \sum_{i=1}^n X_i$ ，期望：

$$E[T] = \sum_{k=0}^{n-1} \frac{n}{n-k} = n \sum_{k=1}^n \frac{1}{k} = nH_n$$

当 n 较大时 $H_n \approx \ln n + \gamma$ ($\gamma \approx 0.577$)，故 $E[T] \approx n \ln n$ 。

1.4.2 用公平硬币生成随机排列

给定一枚公平硬币，设计算法生成 $1, 2, \dots, n$ 的随机排列。

初始化数组 A ： $A[i] \leftarrow i$ 对 $i = 1$ 到 n

for $i \leftarrow 1$ to n :

 用硬币生成一个 j ($1 \leq j \leq i$)

 swap($A[i], A[j]$)

最优情况时间复杂度 $C_{\text{opt}}(n) = O(n \log n)$ 。考虑拒绝概率时，因拒绝而产生的额外代价 $C_{\text{penalty}}(n) = \sum_{i=2}^n w_i \leq 3n = O(n)$ ，因此最坏时间复杂度 $C_{\text{wor}}(n) = O(n \log n)$ 。

1.4.3 矩阵互逆验证

给定两个 $n \times n$ 方阵 A 和 B ，判定是否互逆 ($AB = I$)。基于 Monte Carlo 方法，通过随机采样验证 AB 矩阵元素：

- 对角线元素应为 1（允许误差 ϵ ）
- 非对角线元素应为 0（允许误差 ϵ ）